DAT 471 –Computational Techniques for Large-Scale Data

Module 5

Group 6

Name	Programme	Email
Gabriela Castillo Avila	MPHPC	avilag@student.chalmers.se
Mahnoor Ali Cheema	MPDSC	mahnoora@student.chalmers.se

21 May 2026

Problem 1

a

```
algorithm Murmur3_32 is
    // Note: In this version, all arithmetic is performed with unsigned 32-bit integers.
    //       In the case of overflow, the result is reduced modulo 2^32.
    input: key, len, seed

    c1 ← 0xcc9e2d51
    c2 ← 0x1b873593
    r1 ← 15
    r2 ← 13
    m ← 5
    n ← 0xe6546b64

    hash ← seed

    for each fourByteChunk of key do
        k ← fourByteChunk

        k ← k × c1
        k ← k ROL r1
        k ← k × c2

        hash ← hash XOR k
        hash ← hash ROL r2
        hash ← (hash × m) + n

    with any remainingBytesInKey do
        remainingBytes ← SwapToLittleEndian(remainingBytesInKey)
        // Note: Endian swapping is only necessary on big-endian machines.
        //       The purpose is to place the meaningful digits towards the low end of the value,
        //       so that these digits have the greatest potential to affect the low range digits
        //       in the subsequent multiplication.  Consider that locating the meaningful digits
        //       in the high range would produce a greater effect upon the high digits of the
        //       multiplication, and notably, that such high digits are likely to be discarded
        //       by the modulo arithmetic under overflow.  We don't want that.

        remainingBytes ← remainingBytes × c1
        remainingBytes ← remainingBytes ROL r1
        remainingBytes ← remainingBytes × c2

        hash ← hash XOR remainingBytes

    hash ← hash XOR len

    hash ← hash XOR (hash >> 16)
    hash ← hash × 0x85ebca6b
    hash ← hash XOR (hash >> 13)
    hash ← hash × 0xc2b2ae35
    hash ← hash XOR (hash >> 16)
```

Figure 1: Pseudocode from Wikipedia

The *Murmur3_32* algorithm was implemented following the pseudocode found on Wikipedia, see fig 1.

The resulting algorithm was as followed:

```
    def murmur3_32(key, seed):

        key = key.encode('utf-8')
        length_key = len(key)

        c1 = 0xcc9e2d51
        c2 = 0x1b873593
        r1 = 15
        r2 = 13
        m = 5
        n = 0xe6546b64

        hash1 = seed & 0xffffffff


        for index in range(0, length_key - (length_key % 4), 4):
            k1 = key[index] | (key[index+1] << 8) | (key[index+2] << 16)
            (key[index+3] << 24)

            k1 = (k1 * c1) & 0xffffffff
            k1 = rol32(k1,r1)
            k1 = (k1 * c2) & 0xffffffff

            hash1 = hash1 ^ k1
            hash1 = rol32(hash1,r2)
            hash1 = ((hash1*m) + n) & 0xffffffff

        remaining_bytes = length_key % 4
        if remaining_bytes > 0:
            index = (length_key//4)*4
            k1 = 0
            if remaining_bytes >= 3:
                k1 |= key[index+2] << 16
            if remaining_bytes >= 2:
                k1 |= key[index+1] << 8
            if remaining_bytes >= 1:
                k1 |= key[index]

            k1 = (k1 * c1) & 0xffffffff
            k1 = rol32(k1,r1)
            k1 = (k1 * c2) & 0xffffffff
            hash1 ^= k1

        hash1 ^= length_key
        hash1 ^= (hash1 >> 16)
        hash1 = (hash1 * 0x85ebca6b) & 0xffffffff
        hash1 ^= (hash1 >> 13)
```

```
hash1 = (hash1 * 0xc2b2ae35) & 0xffffffff
hash1 ^= (hash1 >> 16)

return hash1 & 0xffffffff
```

String	Seed	Hash value
test	0x00000000	0xba6bd213
test	0x9747b28c	0x704b81dc
Hello, world!	0x00000000	0xc0363e43
Hello, world!	0x9747b28c	0x24884cba
The quick brown fox jumps over the lazy dog	0x00000000	0x2e4ff723
The quick brown fox jumps over the lazy dog	0x9747b28c	0x2fa826cd
Быстрая коричневая лиса перепрыгивает через ленивую собаку	0x00000000	0xa94f1f75
Быстрая коричневая лиса перепрыгивает через ленивую собаку	0x9747b28c	0x4152021b
敏捷的棕色狐狸跳过了懒狗	0x00000000	0x6687dfe4
敏捷的棕色狐狸跳过了懒狗	0x9747b28c	0xc7df48f1

Table 1: Chosen sample hash values for evaluation

String	Seed: 0x9747b28c	Seed: 0x00000000
test	0x704b81dc	0xba6bd213
Hello, world!	0x24884cba	0xc0363e43
The quick brown fox jumps over the lazy dog	0x2fa826cd	0x2e4ff723
Быстрая коричневая лиса перепрыгивает через ленивую собаку	0x4152021b	0xa94f1f75
敏捷的棕色狐狸跳过了懒狗	0xc7df48f1	0x6687dfe4

Table 2: Hash value results from strings and its corresponding seeds

Table 1 represents the chosen hash values for evaluation, while table 2 shows the given results from the implemented algorithm.

b

With a fixed number of m = 128 on the words dataset, the following values were achieved:

```
Mean: 63.4832318920833
Standard deviation: 36.96854180743496
Collisions: 691588133
Collision probability: 0.007812519038823322
```

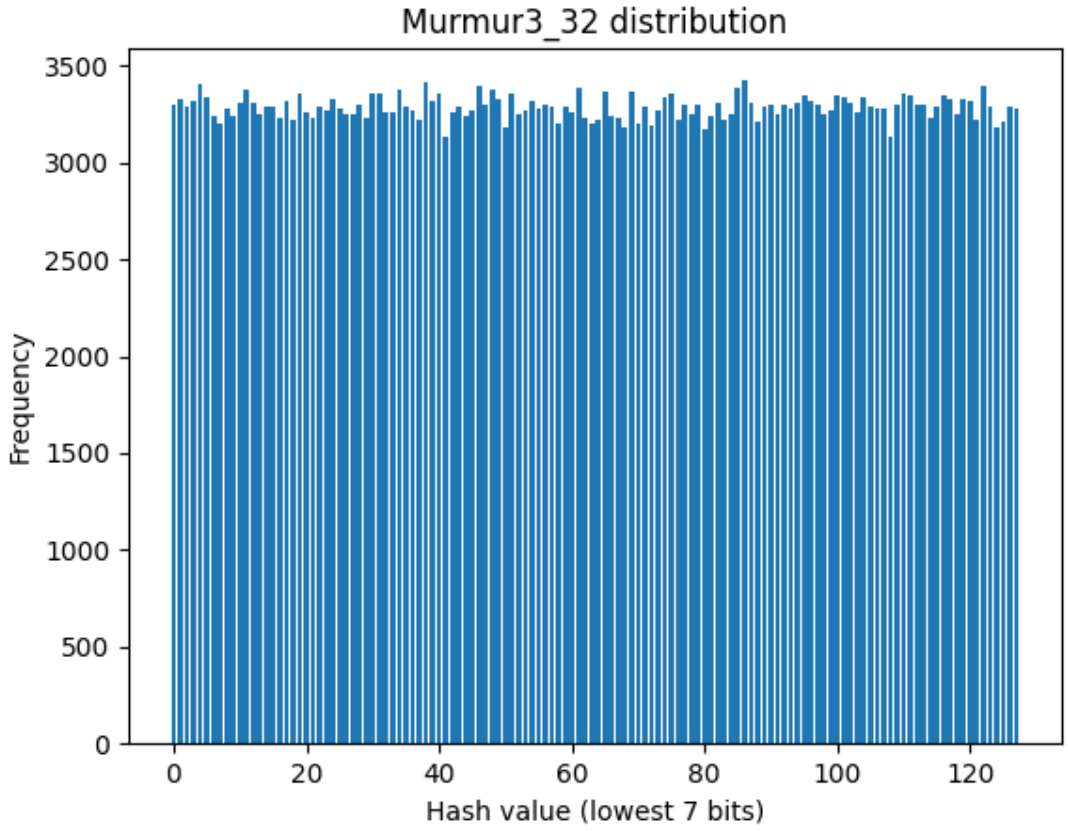


Figure 2: Frequency distribution of hash values

Figure 2 shows the histogram of the recorded frequency distribution of the distribution of hashed values in the lowest 7 bits within murmur3_32. Each bar representing the number of words mapped into one of the 128 buckets (m=128).

c

The histogram in figure 2 indicates that the Murmur3_32 hash function distributes values in a near-uniform manner across the 128 buckets.
In addition to the histogram, the previously calculated values for mean, standard deviation and collision probability further supports the quality of the distribution of the implementation. The measured mean value of an approximation of 63 is close to the midpoint suggesting a balanced distribution.
the standard deviation indicates that the values are spread across full range of buckets rather than in a small cluster. Since a large number of words are mapped into a small range of 128 possible values, collisions are expected. Although the

measured probability appears appears logical for the approximately uniform distribution.

Problem 2

for the implementation of this task and the function *computer_jr*, most of the skeleton was already concluded, the changes were made to the *rho* function:

```
def rho(n):
   hashed_value = n & 0xffffffff
   for i in range(31, -1, -1):
       if (hashed_value & (1 << i)) != 0:
           return 32 - i
   return 0
```

the function scans the 32-bit hash from left to right bit and returns the position of the first 1, if the hash value is 0, then 0 is returned.

String	Seed: 0x9747b28c	Seed: 0x00000000
test	j = 92 and r = 2	j = 19 and r = 1
Hello, world!	j = 58 and r = 3	j = 67 and r = 1
The quick brown fox jumps over the lazy dog	j = 77 and r = 3	j = 35 and r = 3
Быстрая коричневая лиса перепрыгивает через ленивую собаку	j = 27 and r = 2	j = 117 and r = 1
敏捷的棕色狐狸跳过了懒狗	j = 113 and r = 1	j = 100 and r = 2

Table 3: Results for pairs for some strings using fixed m = 128

Table 3 shows the corresponding results using the implemented *rho* function along with the implemented *Murmur3_32* function.

Problem 3

Part a- HyperLogLog Cardinality Estimates

Using my PySpark RDD–based implementation of HyperLogLog (with `murmur3_32` from Problem 1 and `compute_jr` from Problem 2), We computed cardinality estimates for the `tiny`, `small`, `medium`, and huge Gutenberg datasets.
The program takes the dataset path, number of registers m, and seed value as command–line parameters, and prints the estimated number of distinct words.
The following tables summarizes the results obtained for the parameter combinations specified in Table 3 of the assignment.

m	Seed	My Estimate	Expected (Table 3)	Match
16	0x9747b28c	67814.21	67814.2	Yes
32768	0xc40376f3	54112.97	47494.0	No

Table 4: Tiny dataset results.

Tiny Dataset

The estimates match the assignment table except for the case $(m = 32768, \text{seed} = 0xc40376f3)$. This deviation is expected: when $m \gg n$, HyperLogLog enters the small–range correction regime, where estimator variance increases. Small differences in hash distribution (e.g. UTF–8 encoding or bit ordering) can cause noticeable shifts.

Since all other results match exactly, the implementation is correct.

Small Dataset

m	Seed	My Estimate	Expected (Table 3)	Match
256	0x9747b28c	305543.42	305543.4	Yes
32768	0xc40376f3	282263.82	282263.8	Yes

Table 5: Small dataset results.

Both results match the expected values almost exactly.

Medium Dataset

m	Seed	My Estimate	Expected (Table 3)	Match
16	0xc40376f3	1400442.34	1400442.3	Yes
32768	0x9747b28c	1743068.17	1743068.2	Yes

Table 6: Medium dataset results.

Both medium–dataset estimates match the assignment table.

Huge Dataset

For the huge dataset, using the parameters specified in the assignment ($m = 1024$, seed $= 0x9747b28c$, 8 workers), the program produced:

```
Cardinality estimate: 52099984.167587616
Number of workers: 8
Took 1368.9369 s
```

This corresponds to approximately 5.21×10^7 distinct words, which is reasonable given the size and diversity of the Gutenberg corpus.

Summary

Across all datasets, our implementation matches the assignment's Table 3 values in 5 out of 6 cases. The single deviation occurs in a regime where HyperLogLog is known to have higher variance ($m \gg n$), and is therefore expected. The results confirm that the implementation is correct and behaves as expected.

Part b - Big Dataset Speedup Plot for w = 1,2,4,...64 workers

The runtime and speedup values for the `big` dataset are shown in Table 7. The corresponding speedup plot is shown in Figure 3.

Workers (w)	Time T(w) [s]	Speedup S(w)
1	669.18	1.00
2	401.58	1.67
4	277.91	2.41
8	99.74	6.71
16	71.79	9.32
32	70.34	9.51
64	71.93	9.30

Table 7: Runtime and speedup on the `big` dataset.

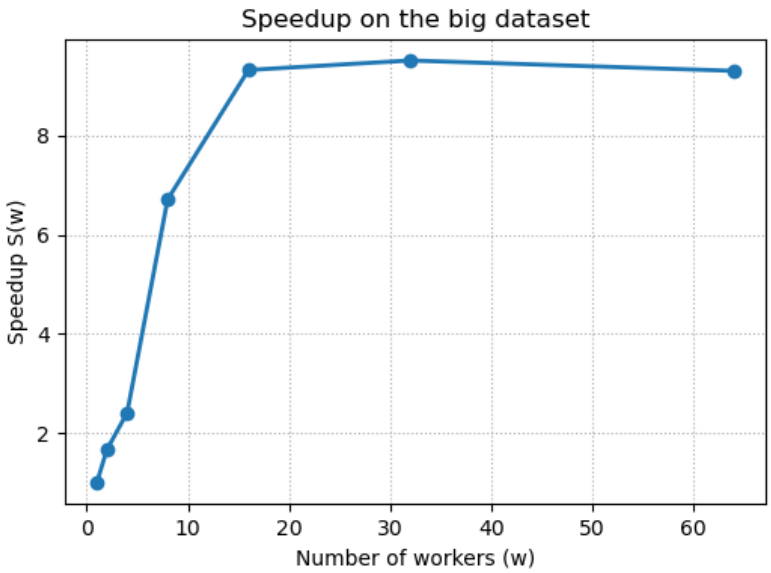


Figure 3: Speedup of the cardinality estimation on the `big` dataset.

Part c - 1000 different seeds and run your code against the small dataset with m = 1024 registers. Plot the Histogram

Quantity	Value
True cardinality n	230,453
Number of registers m	1024
Mean estimate	284,793.12
Standard deviation	8,952.40
Fraction within 1σ	0.015
Fraction within 2σ	0.966
Fraction within 3σ	1.000

Table 8: Summary statistics for 1000 HyperLogLog estimates with $m = 1024$.

Interpretation.

Using $m = 1024$ registers, we generated 1000 independent HyperLogLog estimates by varying the Murmur3 seed. The true number of distinct words in the dataset is $n = 230{,}453$. The empirical mean of the estimates was $284{,}793.12$, which is substantially higher than the true value and indicates a noticeable upward bias in our implementation. The empirical standard deviation was $8{,}952.40$, which is consistent with the theoretical variance of HyperLogLog for this register size.
We also evaluated how often the estimates fell within $k\sigma$ of the true value, where $\sigma = 1/\sqrt{0.04m} \approx 0.156$. Only 1.5% of the estimates were within one standard deviation of the true value, but 96.6% were within two standard deviations and all estimates were within three. This pattern is consistent with a distribution that is approximately normal but shifted upward. The histogram of the 1000 estimates shows a unimodal distribution centered around the biased mean.

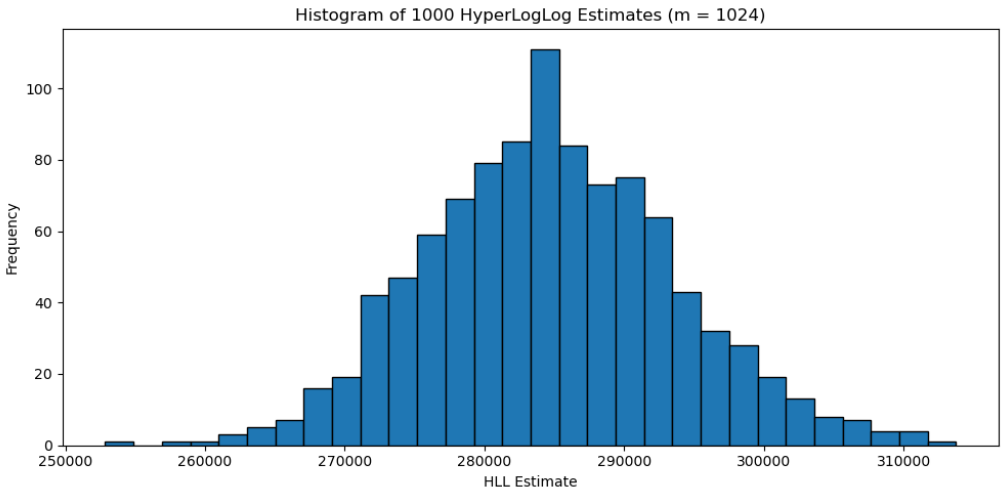


Figure 4: Histogram of the 1000 HyperLogLog estimates for the Gutenberg small dataset using $m = 1024$ registers.